DOSSIER DE TESTS UNITAIRES

Module de gestion des prestataires

Élément	Valeur
Nom du projet	Event Management System
Référence du document	TU-PREST-001
Version du dossier	1.0
Date d’exécution	7 novembre 2026
Branche de travail	hamzakun57-prestataires-setup (basée sur prestataires)
Révision de base	2a3c6ac — changements de test non commités
Statut	Exécuté — 33 tests réussis, 0 échec

Dossier établi sur la base de l’exemple fourni, avec une structure et des scénarios adaptés au code réel du module Prestataires.

1. Historique du document

Version	Date	Intervenant / rôle	Modification	Validation
0.1	07/11/2026	Développement	Création du dossier et définition du périmètre.	À relire
1.0	07/11/2026	Développement / test	Scénarios Node.js, exécution et consignation des résultats.	Résultats exécutés

La validation formelle du responsable de projet, du testeur et de l'encadrant reste à compléter.

2. Objet et objectifs

Ce dossier décrit les tests automatisés du backend Node.js/Express du module Prestataires. Il vérifie les réponses HTTP, la validation des données et les appels SQL préparés en remplaçant le pool PostgreSQL par un faux pool contrôlé. Les tests se rejouent sans écrire dans une base réelle.

Objectifs : vérifier les opérations CRUD des prestataires et services, la recherche par nom/type, les validations et codes d'erreur, la règle de non-suppression si un prestataire est référencé, et le comportement face aux erreurs SQL.

3. Présentation du module et règles couvertes

Le service expose des routes REST sous /api/prestataires. Un prestataire comprend un identifiant, un nom, des coordonnées, un type de service et un tarif. Chaque service associé comprend un nom, une description facultative et un tarif. Les tarifs sont positifs ou nuls et limités par PostgreSQL NUMERIC(12,2).

Règle	Couverture / réserve
RG01	Identifiant généré et unique par la base
RG02	Nom, coordonnées, type de service et tarif obligatoires
RG03	Créer, consulter, modifier et supprimer un prestataire
RG04–RG05	Services liés au prestataire, avec tarif
RG06	Tarif positif/nul, borné à NUMERIC(12,2)
RG07	Recherche filtrée par nom et/ou type de service
RG08	Modification transmise à la base par requête UPDATE
RG09	Suppression refusée si événement/réservation référence le prestataire
RG10	Persistance PostgreSQL structurée; non testée en direct dans cette suite

4. Périmètre et méthode

Outils : Node.js node:test et node:assert/strict, serveur Express sur port éphémère, requêtes HTTP locales via fetch, pool PostgreSQL remplacé par un double de test contrôlant les réponses et enregistrant les paramètres SQL. Chaque scénario prépare le double, appelle une route et vérifie le statut, le JSON et/ou la requête.

Ces tests isolent les routes du vrai PostgreSQL. Ils vérifient le contrat HTTP et la logique des routes, mais ne constituent pas une validation d'intégration de la base ni une mesure de couverture de code.

Exemple détaillé — création d'un prestataire valide

Préparer : injecter un faux pool qui retourne une ligne SQL où id et tarif sont des chaînes. Exécuter : envoyer POST /api/prestataires avec nom, contact, typeService et tarif JSON valides. Vérifier : HTTP 201, nombres convertis dans le JSON, et paramètres SQL avec les chaînes nettoyées. La base réelle n'est pas contactée.

5. Tableau récapitulatif des cas de test

Résultats obtenus lors de l'exécution du 7 novembre 2026. Chaque scénario a réussi.

ID	Scénario	Résultat attendu	Résultat
UT-PRE-01	Rejeter création avec nom vide et champs incomplets	HTTP 400; aucun appel DB	**Réussi**
UT-PRE-02	Rejeter un identifiant non numérique	HTTP 400; aucun appel DB	**Réussi**
UT-PRE-03	Résoudre recherche avant la route paramétrée; convertir les nombres PostgreSQL	HTTP 200; paramètres et modèle corrects	**Réussi**
UT-PRE-04	Refuser la suppression d'un prestataire référencé	HTTP 409; ne pas supprimer	**Réussi**
UT-PRE-05	Consulter le détail et ses services à description nulle	HTTP 200; structure et types corrects	**Réussi**
UT-PRE-06	Créer un prestataire valide et nettoyer les chaînes	HTTP 201; SQL paramétré	**Réussi**
UT-PRE-07	Rendre coordonnées et type de service obligatoires	HTTP 400; aucun appel DB	**Réussi**
UT-PRE-08	Refuser un tarif négatif ou non numérique	HTTP 400; aucun appel DB	**Réussi**
UT-PRE-09	Refuser un tarif supérieur à NUMERIC(12,2)	HTTP 400 avant INSERT	**Réussi**
UT-PRE-10	Accepter zéro et le maximum 9 999 999 999,99	HTTP 201; valeur retournée sans débordement	**Réussi**
UT-PRE-11	Lister les prestataires et mapper les résultats SQL	HTTP 200; liste JSON attendue	**Réussi**
UT-PRE-12	Filtrer par nom et type avec valeurs SQL paramétrées	HTTP 200; filtres normalisés	**Réussi**
UT-PRE-13	Consulter un prestataire absent	HTTP 404 au format erreur	**Réussi**
UT-PRE-14	Modifier un prestataire existant	HTTP 200; UPDATE paramétré	**Réussi**
UT-PRE-15	Modifier un prestataire absent	HTTP 404	**Réussi**
UT-PRE-16	Supprimer un prestataire absent	HTTP 404; arrêt après vérification	**Réussi**
UT-PRE-17	Supprimer un prestataire sans contrat lié	HTTP 204 vide; suppression appelée	**Réussi**
UT-PRE-18	Créer un service d'un prestataire existant	HTTP 201; association et tarif corrects	**Réussi**
UT-PRE-19	Créer un service sans description	HTTP 201; description null	**Réussi**
UT-PRE-20	Refuser un tarif négatif de service	HTTP 400 avant appel DB	**Réussi**
UT-PRE-21	Refuser un tarif de service supérieur à NUMERIC(12,2)	HTTP 400 avant appel DB	**Réussi**
UT-PRE-22	Refuser un service si le prestataire est absent	HTTP 404; aucun INSERT	**Réussi**
UT-PRE-23	Lister les services d'un prestataire existant	HTTP 200; liste convertie	**Réussi**
UT-PRE-24	Lister les services d'un prestataire absent	HTTP 404	**Réussi**
UT-PRE-25	Modifier un service rattaché au bon prestataire	HTTP 200; mise à jour contrainte par les deux ids	**Réussi**
UT-PRE-26	Modifier un service absent ou non rattaché	HTTP 404	**Réussi**
UT-PRE-27	Supprimer un service existant	HTTP 204 vide; ids SQL corrects	**Réussi**
UT-PRE-28	Supprimer un service absent	HTTP 404	**Réussi**
UT-PRE-29	Rejeter un JSON mal formé	HTTP 400 contrôlé	**Réussi**
UT-PRE-30	Répondre de façon homogène à une route inconnue	HTTP 404 JSON	**Réussi**
UT-PRE-31	Mapper une violation de clé étrangère PostgreSQL (23503)	HTTP 409 JSON	**Réussi**

ID	Scénario	Résultat attendu	Résultat
UT-PRE-32	Mapper un dépassement numérique PostgreSQL (22003)	HTTP 400 contrôlé	**Réussi**
UT-PRE-33	Masquer les détails d'une erreur DB inattendue	HTTP 500 générique sans fuite interne	**Réussi**

6. Environnement et procédure d’exécution

Élément	Valeur réellement observée
Runtime	Node.js v24.20.0
Serveur HTTP	Express 4 (service existant)
Framework de test	node:test et node:assert/strict (intégrés à Node.js)
Base pendant les tests	Aucune connexion réelle — faux pool injecté
Fichiers testés	services/prestataires/server.js et services/prestataires/test/server.test.js
Commande exécutée	npm test --prefix services/prestataires -- --test-reporter spec
Branche / base	hamzakun57-prestataires-setup; révision de base 2a3c6ac

Pour rejouer les tests depuis la racine du dépôt :

cd services/prestataires
npm test

7. Bilan d’exécution

Indicateur	Résultat
Tests exécutés	33
Réussis	33
Échoués / ignorés / bloqués	0 / 0 / 0
Taux de réussite des tests exécutés	100 %
Couverture de code instrumentée	Non mesurée

Conclusion : tous les scénarios automatisés réussissent dans l’environnement isolé. Ce résultat ne prouve pas l’accès à PostgreSQL, l’application de la migration, ni la persistance réelle des données. Un test avec une base dédiée reste nécessaire avant validation de bout en bout.

8. Anomalies, limites et validation

Anomalie ciblée : un dépassement de capacité NUMERIC PostgreSQL (SQLSTATE 22003) provoquait une erreur générique lors d’une création. Le backend valide maintenant la limite de NUMERIC(12,2) avant requête et transforme aussi SQLSTATE 22003 en réponse HTTP 400. Ces protections sont couvertes par UT-PRE-09, UT-PRE-10, UT-PRE-21 et UT-PRE-32. Redémarrer l’instance Node qui sert l’API avant de retester.

Limites : le pool SQL est simulé; aucune base de test dédiée n’était configurée. Le comportement lié aux contrats/réservations est vérifié avec des réponses simulées, pas avec une table réelle. Aucun taux de couverture du code n’a été calculé. Le code et le dossier sont des changements locaux non commités à la date du rapport.

Validation formelle (à compléter)

Rôle	Nom	Décision / commentaire	Date / signature
Développeur responsable	À compléter	À compléter	À compléter
Responsable des tests	À compléter	À compléter	À compléter
Chef de projet / encadrant	À compléter	À compléter	À compléter

Annexe — source des tests : services/prestataires/test/server.test.js